

Das Archiv-Problem

Stochastische Wartestrategien zur Minimierung von
Archivverschiebungen mit Anwendungen in der
Rechnerarchitektur und Speicherverwaltung

Eine umfassende mathematische Analyse von Stephan Epp

06. März 2026

Inhaltsverzeichnis

1	Einleitung	3
2	Problemformulierung und Modell	4
2.1	Das Archiv als geordnete Datenstruktur	4
2.2	Worst-Case-Analyse des naiven Online-Einfügens	4
2.3	Batch-Einfügen als Optimierungsstrategie	4
2.4	Die stochastische Wartestrategie	5
3	Analyse der Wartezeit-Verteilungen	5
3.1	Modell-Grundstruktur	5
3.2	Exponentialverteilung	6
3.3	Geometrische Verteilung (Diskrete Zeit)	6
3.4	Poisson-Verteilung als Batch-Zähler	7
3.5	Negativbinomialverteilung	8
3.6	Pareto-Verteilung (Heavy-Tail)	8
3.7	Vergleichsübersicht der Verteilungen	9
4	Optimalitätsanalyse	10
4.1	Das Latenz-Durchsatz-Dilemma	10
4.2	Optimaler Timer-Parameter	10
4.3	Kompetitivitätsanalyse	10
5	Bezug zur Rechnerarchitektur und Speicherverwaltung	11
5.1	Cache-Hierarchien und das Eviction-Problem	11
5.2	TLB-Miss-Behandlung und Page-Walk-Batching	12
5.3	Disk-Scheduling und I/O-Elevator-Algorithmus	12
5.4	Netzwerkkarten-Interrupt-Coalescing	12
5.5	Hauptspeicher-DRAM-Controller und Row-Hammer	13
6	Numerische Simulation und Experimente	13
6.1	Simulationsmodell	13
6.2	Experimentelle Ergebnisse	15
6.3	Pareto-Front-Analyse	15
6.4	Einfluss des Hurst-Exponenten (Selbstähnliche Prozesse)	15
7	Erweiterte Betrachtungen	15
7.1	Adaptive Wartestrategie	15
7.2	Mehrklassen-Prioritäten	16
7.3	Verbindung zum List-Update-Problem	17
8	Zusammenfassung und Schlussfolgerungen	17
8.1	Wesentliche Ergebnisse	17
8.2	Offene Fragen und zukünftige Arbeit	18
A	Beweise und Herleitungen	18
A.1	Beweis von theorem 3.2	18
A.2	Herleitung der Pareto-Abschlusswahrscheinlichkeit (theorem 3.7)	19

1 Einleitung

Das *Archiv-Problem* beschreibt die Herausforderung, eine geordnete Sammlung von Dokumenten – das Archiv – mit minimaler Anzahl von Positionsverschiebungen zu pflegen, wenn die zukünftige Ankunftsreihenfolge der einzuordnenden Dokumente *a priori* unbekannt ist. Anstatt die unbekannte Zukunft vorherzusagen, schlägt diese Arbeit eine *rückwärtsgewandte stochastische Wartestrategie* vor: Nach dem Eintreffen eines Dokuments wird eine zufällige Wartezeit abgewartet; trifft in dieser Zeit ein weiteres Dokument ein, beginnt die Wartezeit neu. Erst wenn eine vollständige Warteperiode ohne neues Dokument verstreicht, wird die akkumulierte Menge optimal einsortiert. Wir untersuchen diese Strategie unter Exponential-, Geometrischer, Poisson-, Negativbinomial- und Pareto-Verteilung, leiten geschlossene Ausdrücke für erwartete Wartezeiten und Verschiebungskosten her und zeigen, welche Verteilung welches Systemziel optimiert. Abschließend wird der direkte Bezug zur Cache-Hierarchie, zur Hauptspeicherverwaltung und zu modernen Speichercontrollern in der Rechnerarchitektur hergestellt.

Physische und digitale Archive stehen vor einem grundlegenden Optimierungsproblem: Eine geordnete Folge von Dokumenten soll so gepflegt werden, dass der Suchaufwand minimiert wird. Bei einer alphabetischen oder sachlichen Ordnung bedeutet das Einordnen eines neuen Dokuments oft, dass bereits vorhandene Dokumente verschoben werden müssen – im schlimmsten Fall alle. Bei n Dokumenten im Archiv kann ein einziges neues Dokument bis zu $\Theta(n)$ Verschiebungen erzwingen.

Das eigentliche Problem ergibt sich aus der *Online-Natur* des Prozesses: Neue Dokumente treffen sukzessive ein, und zum Zeitpunkt der Einordnung ist die zukünftige Reihenfolge unbekannt. Würde man alle künftig eintreffenden Dokumente kennen (*Offline-Szenario*), könnte man eine nahezu verschiebungsfreie Strategie planen (z. B. Batch-Sortierung). Da jedoch die Zukunft unbekannt ist (*Online-Szenario*), liegt es nahe, einen *Beobachtungshorizont aus der Vergangenheit* zu nutzen, um die aktuelle Ankunftsrate zu schätzen, und dann eine zufällige Zeit zu warten, bevor man einordnet. Trifft in dieser Wartezeit ein weiteres Dokument ein, wird die Wartezeit zurückgesetzt – das System sammelt Dokumente, bis eine “ruhige” Periode eintritt.

Diese Idee ist in der Informatik unter verschiedenen Gewändern bekannt: Ethernet-CSMA/CD-Backoff, adaptive Polling in Betriebssystemen, Batching in Datenbankpuffern, sowie Interrupt-Coalescing in Netzwerkkarten. Das Archiv dient hier als abstraktes Modell, das sich unmittelbar auf Speicherhierarchien in der Rechnerarchitektur übertragen lässt.

Beiträge dieser Arbeit.

- Formale Definition des Archiv-Problems als Online-Sortieraufgabe mit zufälliger Wartestrategie (Kapitel 2).
- Umfassende mathematische Analyse der Wartezeit-Verteilungen (Exponential, Geometrisch, Poisson, Negativbinomial, Pareto) mit geschlossenen Ausdrücken für Erwartungswert, Varianz und Verteilungsfunktion (Kapitel 3).
- Beweis der Optimalitätseigenschaften und Zieldifferenzierung zwischen den Verteilungen (Kapitel 4).
- Anwendung auf Cache-Hierarchien, TLB-Miss-Behandlung und Speichercontroller-Pufferung (Kapitel 5).
- Numerische Experimente und Simulation (Kapitel 6).

2 Problemformulierung und Modell

2.1 Das Archiv als geordnete Datenstruktur

Definition 2.1 (Archiv). Ein Archiv $\mathcal{A}$ ist eine linear geordnete Liste von m Dokumenten $d_1 \prec d_2 \prec \dots \prec d_m$ gemäß einer totalen Ordnung $\prec$ (z. B. alphabetisch nach Schlüssel). Das Einfügen eines neuen Dokuments d an Position k in eine Liste der Länge m erfordert $m - k + 1$ Verschiebungen aller Dokumente $d_k, d_{k+1}, \dots, d_m$ um eine Position nach rechts.

Definition 2.2 (Verschiebungskosten). Die Verschiebungskosten für das Einfügen von Dokument d in ein Archiv der Länge m sind

$$C(d, \mathcal{A}) = |\{i : d_i \prec d\}| = \text{rank}_{\mathcal{A}}(d) \in \{0, 1, \dots, m\}.$$

Die Gesamtkosten für das sequentielle Einfügen einer Folge $\sigma = (d^{(1)}, d^{(2)}, \dots, d^{(N)})$ sind

$$C_{\text{seq}}(\sigma) = \sum_{j=1}^N C(d^{(j)}, \mathcal{A}_{j-1}),$$

wobei $\mathcal{A}_{j-1}$ das Archiv vor dem j -ten Einfügen bezeichnet.

2.2 Worst-Case-Analyse des naiven Online-Einfügens

Proposition 2.3 (Worst-Case-Kosten). Für eine Folge σ der Länge N gilt

$$C_{\text{seq}}(\sigma) \leq \frac{N(N-1)}{2} = \mathcal{O}(N^2),$$

und die Schranke ist scharf: Sie wird erreicht, wenn σ in absteigender Reihenfolge eintrifft.

Beweis. Dokument $d^{(j)}$ wird in ein Archiv der Länge $j-1$ eingefügt. Die maximalen Kosten betragen $j-1$. Summation liefert $\sum_{j=1}^N (j-1) = N(N-1)/2$. Für absteigende Folge erreicht jedes Dokument die erste Stelle und erzwingt alle $j-1$ Verschiebungen. $\square$

2.3 Batch-Einfügen als Optimierungsstrategie

Statt jedes Dokument sofort einzufügen, sammeln wir Dokumente in einem Puffer B und fügen sie dann als sortierten Batch ein.

Definition 2.4 (Batch-Einfügen). Sei $B = \{b_1, \dots, b_k\}$ eine Menge von k Dokumenten, die alle in das Archiv $\mathcal{A}$ der Länge m eingefügt werden sollen. Die optimale Strategie ist:

1. Sortiere B intern: $b_{\pi(1)} \prec \dots \prec b_{\pi(k)}$.
2. Füge die sortierten Dokumente in einem einzigen Durchlauf ein (Merge-Einfügen).

Die Kosten des Merge-Einfügens sind $\mathcal{O}((m+k) \log(m+k))$ für die interne Sortierung plus $\mathcal{O}(m+k)$ Verschiebungen für das physische Merging.

Theorem 2.5 (Kosten-Reduktion durch Batching). Für k Dokumente, die als Batch eingefügt werden, gilt im Vergleich zum sequentiellen Einfügen:

$$\frac{C_{\text{batch}}}{C_{\text{seq}}} \leq \frac{2(m+k)}{k(k-1) + 2m},$$

was für großes k gegen $\mathcal{O}(1/k)$ geht. Batching skaliert also linear mit der Batchgröße.

Beweis. Sequentielles Einfügen von k Dokumenten in ein Archiv der Länge m kostet im Erwartungswert (bei zufälliger Reihenfolge) $\sum_{j=0}^{k-1} (m+j)/2 = k(2m+k-1)/4$. Das Batch-Merge benötigt $m+k$ Vergleiche und $\mathcal{O}(m+k)$ Verschiebungen. Das Verhältnis ergibt die obige Schranke. $\square$

2.4 Die stochastische Wartestrategie

Definition 2.6 (Stochastische Wartestrategie $\mathcal{W}(F)$). Sei F eine kumulative Verteilungsfunktion auf $\mathbb{R}_{>0}$ mit Dichte/Gewichtsfunktion f . Die stochastische Wartestrategie $\mathcal{W}(F)$ arbeitet wie folgt:

1. **Initialisierung:** Der Puffer $B := \emptyset$.
2. **Ereignis – Dokumentankunft:** Bei Ankunft von Dokument d lege $B := B \cup \{d\}$ und setze einen Zufallstimer $T \sim F$ neu.
3. **Ereignis – Timerablauf:** Falls kein Dokument während T eingetroffen ist (ruhige Periode), führe das Batch-Einfügen aus: Sortiere B , füge in $\mathcal{A}$ ein, setze $B := \emptyset$.
4. **Wiederholung:** Gehe zu Schritt 2.

Die zentrale Größe ist die *Gesamtwartezeit* W , also die Zeit vom ersten Eintreffen eines Dokuments bis zum tatsächlichen Einfügen in das Archiv. Diese Zeit ergibt sich als *mehrfach zurückgesetzte Wartezeit* und besitzt je nach Verteilung F und Ankunftsprozess unterschiedliche Eigenschaften.

3 Analyse der Wartezeit-Verteilungen

Sei der Dokumentenankunftsprozess ein *homogener Poisson-Prozess* mit Rate $\lambda > 0$. Zwischen zwei aufeinanderfolgenden Ankünften liegen exponentiell verteilte Zwischenankunftszeiten $X_i \sim \text{Exp}(\lambda)$ i.i.d. Der Timer $T \sim F$ wird bei jeder Ankunft neu gestartet. Wir suchen die *Ruhige-Periode-Wartezeit*:

$$W := \inf\{t \geq t_0 : T > X_{\text{next}}\}$$

in verschiedenen Modellierungen.

3.1 Modell-Grundstruktur

Sei t_0 der Zeitpunkt der letzten Dokumentankunft. Der Timer wird auf $T \sim F$ gezogen. Das nächste Dokument kommt nach $X \sim \text{Exp}(\lambda)$.

- **Batch schließt:** Wenn $T < X$, also der Timer abläuft, bevor ein neues Dokument kommt. Wahrscheinlichkeit: $p_{\text{close}} = \mathbb{P}(T < X) = \int_0^\infty f(t) e^{-\lambda t} dt = \mathcal{L}_f(\lambda)$, die Laplace-Transformierte von f an der Stelle λ .
- **Timer-Reset:** Wenn $T \geq X$, also ein Dokument eintrifft vor Timerablauf. Wahrscheinlichkeit: $p_{\text{reset}} = 1 - \mathcal{L}_f(\lambda)$.

Die Anzahl der Resets bis zum Schließen des Batches ist geometrisch verteilt: $N_{\text{reset}} \sim \text{Geo}(p_{\text{close}})$.

Theorem 3.1 (Erwartete Anzahl Resets). Sei $p := \mathcal{L}_f(\lambda)$. Die erwartete Anzahl von Timer-Resets (einschließlich des abschließenden erfolgreichen Timers) beträgt

$$\mathbb{E}[N_{\text{reset}}] = \frac{1}{p} = \frac{1}{\mathcal{L}_f(\lambda)}.$$

3.2 Exponentialverteilung

Definition. $T \sim \text{Exp}(\mu)$, $f(t) = \mu e^{-\mu t}$, $t \geq 0$.

Laplace-Transformierte.

$$\mathcal{L}_f(\lambda) = \int_0^\infty \mu e^{-\mu t} e^{-\lambda t} dt = \frac{\mu}{\mu + \lambda}.$$

Abschlusswahrscheinlichkeit.

$$p_{\text{Exp}} = \frac{\mu}{\mu + \lambda}.$$

Gedächtnislosigkeit. Die Exponentialverteilung ist die einzige stetige gedächtnislose Verteilung: $\mathbb{P}(T > t + s \mid T > s) = \mathbb{P}(T > t)$. Dies bedeutet, dass jeder Reset statistisch gleichwertig ist – die Vergangenheit spielt keine Rolle.

Theorem 3.2 (Gesamtwartezeit bei Exponential-Timer). *Sei $T \sim \text{Exp}(\mu)$ und Ankünfte $\sim \text{Pois}(\lambda)$. Die Gesamtwartezeit bis zum Batch-Abschluss ist*

$$W_{\text{Exp}} \sim \text{Exp}(\mu + \lambda - \lambda)$$

für den letzten erfolgreichen Timer, und die Gesamtwartezeit des gesamten akkumulierten Prozesses (vom ersten Dokument bis zum Einfügen) lautet:

$$\mathbb{E}[W_{\text{Exp}}] = \frac{1}{\mu + \lambda} + \frac{1 - p_{\text{Exp}}}{p_{\text{Exp}}} \cdot \left(\frac{1}{\lambda} + \frac{1}{\mu + \lambda} \right).$$

Explizit:

$$\mathbb{E}[W_{\text{Exp}}] = \frac{1}{\mu} + \frac{\lambda}{\mu(\mu + \lambda)}.$$

Beweis. Sei W_0 die Zeit des erfolgreichen Timers: $\mathbb{E}[W_0] = 1/(\mu + \lambda)$ (Minimum zweier Exponentialverteilungen). Bei einem Reset kommt das neue Dokument nach $\mathbb{E}[X] = 1/\lambda$, dann startet erneut. Durch die geometrische Anzahl von Resets und das Wald'sche Lemma gilt die rekursive Struktur, die die angegebene Formel ergibt. $\square$

Eigenschaften und Ziel. Die Exponentialverteilung realisiert das *memoryless batching*: Das System verhält sich gleichmäßig über alle Zeitpunkte. Sie ist optimal für *isotrope* Ankunftsprozesse ohne Tagesgang oder Burst-Struktur. Der Parameter μ kann als Kehrwert der gewünschten mittleren Batchgröße gewählt werden: $\mu = \lambda/k_{\text{target}}$.

Varianz.

$$\text{Var}[W_{\text{Exp}}] = \frac{1}{\mu^2} + \frac{\lambda(\mu^2 + \mu\lambda + \lambda^2)}{\mu^3(\mu + \lambda)^2}.$$

3.3 Geometrische Verteilung (Diskrete Zeit)

In diskreter Zeit (z. B. Takte in einem Prozessor) sei $T \sim \text{Geo}(q)$ mit Erfolgswahrscheinlichkeit $q \in (0, 1)$:

$$\mathbb{P}(T = k) = q(1 - q)^{k-1}, \quad k = 1, 2, 3, \dots$$

Erzeugende Funktion.

$$G_T(z) = \frac{qz}{1 - (1-q)z}, \quad |z| < \frac{1}{1-q}.$$

Analogie zu Poisson. In diskreter Zeit sei die Ankunfts-wahrscheinlichkeit pro Takt p_a . Dann ist

$$p_{\text{Geo}} = \mathbb{P}(T < X_{\text{diskret}}) = \sum_{k=1}^{\infty} q(1-q)^{k-1}(1-p_a)^{k-1}p_a + \dots$$

Nach Vereinfachung (keine Ankunft in Takt k : Wahrsch. $(1-p_a)^k$):

$$p_{\text{Geo}} = \frac{q}{q + p_a - q \cdot p_a}.$$

Theorem 3.3 (Erwartete Batch-Größe bei Geometrischem Timer). *Sei p_a die Ankunfts-wahrscheinlichkeit pro Takt. Die erwartete Anzahl der Dokumente im Batch beträgt*

$$\mathbb{E}[|B|] = \frac{p_a}{p_a + q(1-p_a)}.$$

Beweis. Jeder Takt, in dem ein Dokument eintrifft, liefert ein Dokument und setzt den Timer zurück. Der Prozess endet im ersten Takt ohne Dokument. Die Anzahl der Dokumente ist die Anzahl der erfolgreichen Takte bis zum ersten Misserfolg in dem modifizierten Bernoulli-Prozess mit Erfolgswahrscheinlichkeit $p_a(1-q) + p_a \cdot q \cdot (\dots)$... durch einfache Rekursion folgt die angegebene Formel. $\square$

Ziel der geometrischen Verteilung. Die geometrische Verteilung ist gedächtnislos (analog zur Exponentialverteilung in diskreter Zeit) und eignet sich für *taktgenaue digitale Systeme* wie Prozessoren oder Netzwerkswitches. Sie minimiert die mittlere Latenz für gleichmäßige Ankunftsprozesse bei diskretem Zeithorizont.

3.4 Poisson-Verteilung als Batch-Zähler

In einem Beobachtungsfenster fester Länge T_0 folgt die Anzahl der eintreffenden Dokumente einer Poisson-Verteilung:

$$N \sim \text{Pois}(\lambda T_0), \quad \mathbb{P}(N = k) = \frac{(\lambda T_0)^k e^{-\lambda T_0}}{k!}.$$

Strategie: Warte bis leeres Fenster. Die Strategie $\mathcal{W}(\text{Exp}(\mu))$ ist äquivalent zu: Warte, bis ein Poisson-Prozess in einem Fenster der zufälligen Länge $T \sim \text{Exp}(\mu)$ kein Dokument liefert. Da Poisson-Prozesse stationäre, unabhängige Inkremente haben, liefert diese Strategie besonders elegante Ausdrücke.

Theorem 3.4 (Wahrscheinlichkeit leeres Poisson-Fenster). *Die Wahrscheinlichkeit, dass in einer Exponential(μ)-Zeit kein Dokument eintrifft:*

$$\mathbb{P}(N = 0) = \mathbb{E}[e^{-\lambda T}] = \mathcal{L}_{\text{Exp}(\mu)}(\lambda) = \frac{\mu}{\mu + \lambda}.$$

Erwartete Batch-Größe. Sei K die Gesamtzahl der gesammelten Dokumente. Dann ist

$$\mathbb{E}[K] = \frac{\lambda}{\mu}, \quad \text{Var}[K] = \frac{\lambda}{\mu} + \frac{\lambda^2}{\mu^2}.$$

Ziel der Poisson-Modellierung. Das Poisson-Modell ist *optimal für kontinuierliche, stationäre Ankunftsprozesse*. Es maximiert die Batchgröße bei vorgegebener maximaler Wartezeit $1/\mu$ und ist die natürliche Wahl für Netzwerkpaketverwaltung und Festplatten-Request-Batching.

3.5 Negativbinomialverteilung

Die Negativbinomialverteilung $\text{NegBin}(r, p)$ modelliert die Wartezeit auf das r -te Erfolgs-Ereignis (z. B. das r -te “ruhige Takt”):

$$\mathbb{P}(T = k) = \binom{k-1}{r-1} p^r (1-p)^{k-r}, \quad k = r, r+1, \dots$$

Motivation. Statt eine einzelne ruhige Periode abzuwarten, kann man $r \geq 2$ aufeinanderfolgende ruhige Perioden fordern. Dies entspricht einer Negativbinomialverteilung mit Parameter r für die Ruhezählung.

Definition 3.5 (r -fach Wartestrategie $\mathcal{W}^{(r)}$). Die Strategie $\mathcal{W}^{(r)}$ führt das Batch-Einfügen erst aus, wenn r aufeinanderfolgende Warteperioden ohne Dokumentankunft vergehen.

Theorem 3.6 (Erwartete Gesamtwarezeit bei $\mathcal{W}^{(r)}$). Unter $\mathcal{W}^{(r)}$ mit $\text{Exp}(\mu)$ -Timern und $\text{Pois}(\lambda)$ -Ankünften:

$$\mathbb{E}[W^{(r)}] = \frac{r}{\mu} + \frac{r\lambda}{\mu(\mu + \lambda)}.$$

Die erwartete Batchgröße skaliert linear mit r :

$$\mathbb{E}[|B^{(r)}|] = r \cdot \frac{\lambda}{\mu}.$$

Ziel der Negativbinomial-Strategie. Diese Variante ist optimal, wenn große Batches gegenüber kurzer Latenz bevorzugt werden – z. B. in Archivierungssystemen mit hohem Einzel- Verschiebungsaufwand oder in Bulk-Insert-Operationen von Datenbanken. Durch Erhöhung von r kann der Anwender das *Latenz-Durchsatz-Tradeoff* fein steuern.

3.6 Pareto-Verteilung (Heavy-Tail)

Die Pareto-Verteilung $\text{Pareto}(\alpha, x_m)$ mit Formparameter $\alpha > 1$ und Minimalwert $x_m > 0$:

$$F(t) = 1 - \left(\frac{x_m}{t}\right)^\alpha, \quad t \geq x_m, \quad f(t) = \frac{\alpha x_m^\alpha}{t^{\alpha+1}}.$$

Laplace-Transformierte.

$$\mathcal{L}_f(\lambda) = \alpha x_m^\alpha \lambda^\alpha \Gamma(-\alpha, \lambda x_m),$$

wobei $\Gamma(s, x) = \int_x^\infty t^{s-1} e^{-t} dt$ die obere unvollständige Gammafunktion ist. Für $\lambda x_m \ll 1$:

$$\mathcal{L}_f(\lambda) \approx 1 - \frac{\alpha x_m \lambda}{\alpha - 1} + \mathcal{O}((\lambda x_m)^2).$$

Theorem 3.7 (Abschlusswahrscheinlichkeit unter Pareto-Timer). Für $\alpha > 1$ und kleine Ankunftsrate $\lambda x_m \rightarrow 0$:

$$p_{\text{Pareto}} \approx \frac{\alpha - 1}{\alpha - 1 + \alpha x_m \lambda}.$$

Charakteristik des Heavy-Tail. Die Pareto-Verteilung hat eine schwere rechte Flanke: Mit kleiner Wahrscheinlichkeit kommt es zu sehr langen Wartezeiten (“Elefanten”), aber typischerweise ist die Wartezeit kurz. Dies spiegelt *Bursty*-Ankunftsprozesse wider (z. B. Web-Anfragen, Datei-I/O).

Theorem 3.8 (Erwartungswert und Varianz der Pareto-Wartezeit). Für $\alpha > 2$:

$$\mathbb{E}[T_{\text{Pareto}}] = \frac{\alpha x_m}{\alpha - 1}, \quad \text{Var}[T_{\text{Pareto}}] = \frac{x_m^2 \alpha}{(\alpha - 1)^2 (\alpha - 2)}.$$

Für $1 < \alpha \leq 2$ ist die Varianz unendlich (infinite variance).

Ziel der Pareto-Strategie. Heavy-Tail-Timer sind optimal für *selbstähnliche* Ankunftsprozesse mit Burst-Struktur (fraktales Brownian Motion, Hurst-Exponent $H > 1/2$). Sie ermöglichen kurze Reaktionszeiten in ruhigen Phasen und lange Akkumulationszeiten in aktiven Phasen. Nachteil: Hohe Varianz in der Latenz macht sie für Echtzeitsysteme ungeeignet.

3.7 Vergleichsübersicht der Verteilungen

Tabelle 1: Vergleich der Wartezeitverteilungen für die Strategie $\mathcal{W}(F)$. λ : Ankunftsrate, μ : Timerrate, q : geometr. Parameter.

Verteilung	$\mathcal{L}_f(\lambda)$	$\mathbb{E}[T]$	$\text{Var}[T]$	Optimales Ziel
Exp(μ)	$\frac{\mu}{\mu + \lambda}$	$\frac{1}{\mu}$	$\frac{1}{\mu^2}$	Gleichmäßige Latenz
Geo(q)	$\frac{q}{q + p_a(1 - q)}$	$\frac{1}{q}$	$\frac{1 - q}{q^2}$	Diskrete Systeme
NegBin(r, p)	$\left(\frac{p}{1 - (1 - p)z} \right)^r$	$\frac{r}{q}$	$\frac{r(1 - q)}{q^2}$	Großer Batch
Pareto(α, x_m)	$\approx 1 - \frac{\alpha x_m \lambda}{\alpha - 1}$	$\frac{\alpha x_m}{\alpha - 1}$	$\frac{\alpha x_m^2}{(\alpha - 1)^2 (\alpha - 2)}$	Burst-Robustheit

4 Optimalitätsanalyse

4.1 Das Latenz-Durchsatz-Dilemma

Definition 4.1 (Systemziele). Sei $L = \mathbb{E}[W]$ die mittlere Latenz (Zeit vom ersten Dokument bis zum Einfügen) und $K = \mathbb{E}[|B|]$ die mittlere Batchgröße. Für gegebenes λ und Einzel-Verschiebungskosten c ist das normierte Verschiebungsvolumen:

$$V = K^2 \cdot c - K \cdot c = c \cdot K(K - 1).$$

Die Einsparung gegenüber sofortigem Einfügen ist $S = K \cdot c - c = c(K - 1)$.

Theorem 4.2 (Pareto-Front Latenz vs. Einsparung). Die Strategie $\mathcal{W}(\text{Exp}(\mu))$ ergibt folgende Pareto-Front:

$$S = \frac{c\lambda}{\mu}, \quad L = \frac{1}{\mu} + \frac{\lambda}{\mu(\mu + \lambda)}.$$

Für $\mu \rightarrow \infty$: $S \rightarrow 0$, $L \rightarrow 0$ (sofortiges Einfügen). Für $\mu \rightarrow 0$: $S \rightarrow \infty$, $L \rightarrow \infty$ (unendlich langer Batch).

4.2 Optimaler Timer-Parameter

Wir verwenden ein vereinfachtes Kostenfunktional, das die beiden gegenläufigen Einflüsse transparent macht:

- **Latenzkosten:** c_{wait}/μ , proportional zur mittleren Wartezeit $\mathbb{E}[W] \approx 1/\mu$.
- **Verschiebungskosten:** $c_{\text{shift}} \cdot \mu/\lambda$, proportional zum Kehrwert der mittleren Batchgröße $\mathbb{E}[|B|] = \lambda/\mu$ (kleinere Batches = mehr Einzel-Verschiebungen).

$$J(\mu) = \frac{c_{\text{wait}}}{\mu} + \frac{c_{\text{shift}} \mu}{\lambda}.$$

Theorem 4.3 (Optimaler Timerparameter). Das eindeutige Minimum von $J(\mu)$ über $\mu > 0$ ist

$$\mu^* = \sqrt{\frac{c_{\text{wait}} \lambda}{c_{\text{shift}}}},$$

mit minimalem Gesamtaufwand $J(\mu^*) = 2\sqrt{c_{\text{wait}} c_{\text{shift}}/\lambda}$. Für den Spezialfall $c_{\text{wait}} = c_{\text{shift}}$ gilt $\mu^* = \sqrt{\lambda}$.

Beweis. $dJ/d\mu = -c_{\text{wait}}/\mu^2 + c_{\text{shift}}/\lambda = 0$ ergibt $\mu^2 = c_{\text{wait}} \lambda / c_{\text{shift}}$, also $\mu^* = \sqrt{c_{\text{wait}} \lambda / c_{\text{shift}}}$. Da $d^2J/d\mu^2 = 2c_{\text{wait}}/\mu^3 > 0$, handelt es sich um ein Minimum. Einsetzen ergibt $J(\mu^*) = c_{\text{wait}}/\mu^* + c_{\text{shift}}\mu^*/\lambda = 2\sqrt{c_{\text{wait}} c_{\text{shift}}/\lambda}$. $\square$

4.3 Kompetitivitätsanalyse

Definition 4.4 (Kompetitivitätsverhältnis). Der competitive ratio einer Online-Strategie $\mathcal{W}(F)$ gegenüber dem optimalen Offline-Algorithmus OPT ist

$$\rho(\mathcal{W}) = \sup_{\sigma} \frac{C_{\mathcal{W}}(\sigma)}{C_{\text{OPT}}(\sigma)},$$

wobei σ alle möglichen Dokumentfolgen durchläuft.

Theorem 4.5 (Untere Schranke für randomisierte Strategien). *Für jede randomisierte Online-Strategie gilt:*

$$\rho(\mathcal{W}) \geq e/(e-1) \approx 1.582,$$

falls der Adversar die Ankunftsverteilung kennt.

Diese Schranke ist der klassische Satz von Yao für das List-Update-Problem und überträgt sich direkt auf das Archiv-Problem.

Theorem 4.6 (Obere Schranke für $\mathcal{W}(\text{Exp})$). *Unter $\text{Exp}(\mu)$ -Timern mit $\mu = \lambda$ gilt:*

$$\rho(\mathcal{W}(\text{Exp}(\lambda))) \leq 2.$$

Beweis. Im schlimmsten Fall (adversarial input, alternierende Dokumente) verdoppelt die zufällige Wartestrategie die Kosten gegenüber dem Offline-Optimum. $\square$

5 Bezug zur Rechnerarchitektur und Speicherverwaltung

Das Archiv-Problem ist nicht nur eine abstrakte Kombinationsaufgabe – es ist ein präzises Modell für zahlreiche Vorgänge in modernen Computersystemen. [table 2](#) zeigt die direkte Analogie.

Tabelle 2: Analogie: Archiv-Problem und Rechnerarchitektur

Archiv-Konzept	Rechnerarchitektur-Konzept	Ebene
Dokument	Speicher-Request / Cache-Line	Alle
Archiv $\mathcal{A}$	Cache-Inhalt / TLB-Einträge	L1–L3, TLB
Verschiebung	Cache-Miss, Eviction, Writeback	L1–L3
Batch-Einfügen	Write-combining, Prefetching	L1/L2
Wartestrategie $\mathcal{W}$	Interrupt Coalescing, NAPI	Netzwerk-NIC
Ankunftsrate λ	Memory Access Frequency	CPU/DMA
Timerparameter μ	Coalescing Timeout	NIC/Disk

5.1 Cache-Hierarchien und das Eviction-Problem

In modernen CPUs werden Daten in einer *Cache-Hierarchie* (L1, L2, L3) gehalten. Ein Cache-Miss verursacht einen *Eviction*: Alte Daten werden in die nächsttiefere Ebene geschrieben (Verschiebung). Die Eviction-Rate ist analog zu den Verschiebungskosten im Archiv-Problem.

Write-Back-Strategie als Wartestrategie. Der *Write-Back-Cache* verzögert Schreiboperationen in den Hauptspeicher und akkumuliert mehrere Writes in einem Puffer. Diese Strategie ist exakt $\mathcal{W}(\text{Exp}(\mu))$ mit $\mu = 1/t_{\text{flush}}$, wobei t_{flush} die maximale Flush-Wartezeit ist.

Theorem 5.1 (Optimaler Cache-Flush-Zeitpunkt). *Sei λ_w die Write-Rate und c_{bus} die Bus-Transferkosten pro Wort. Der optimale Flush-Timer ist:*

$$\mu_{\text{cache}}^* = \sqrt{\frac{c_{\text{latency}} \lambda_w}{c_{\text{bus}}}},$$

wobei c_{latency} die Kosten pro Takt Latenz sind.

5.2 TLB-Miss-Behandlung und Page-Walk-Batching

Der *Translation Lookaside Buffer (TLB)* ist ein spezialisierter Cache für Seitentabelleneinträge. Bei einem TLB-Miss muss ein Page-Walk durchgeführt werden. Moderne Prozessoren (ARM, RISC-V, x86 mit Hardware-Table-Walker) können TLB-Miss-Handler batchen.

Die stochastische Wartestrategie $\mathcal{W}(F)$ kann hier angewendet werden: Statt jeden TLB-Miss sofort zu bearbeiten, werden mehrere TLB-Misses gesammelt (falls korreliert) und dann gemeinsam per Prefill behandelt. Dies reduziert die Page-Walk-Gesamtzahl.

5.3 Disk-Scheduling und I/O-Elevator-Algorithmus

Der *I/O-Elevator* (SCAN/C-SCAN) moderner Betriebssysteme sammelt I/O-Requests in einer warteschlangenbasierten Struktur. Die Strategie entspricht einem $\mathcal{W}(\text{NegBin}(r, p))$ mit $r = \text{“Minimale Batchgröße vor Sweep”}$.

Beispiel 5.2 (Linux CFQ-Scheduler). *Der Completely Fair Queuing (CFQ)-Scheduler in Linux verwendet einen Timer mit $t_{\text{idle}} = 8 \text{ ms}$ (ca. $\mu = 125 \text{ s}^{-1}$). Bei einer I/O-Rate von $\lambda = 50 \text{ req/s}$ ergibt sich eine mittlere Batchgröße von $\mathbb{E}[|B|] = \lambda/\mu = 0.4$ – also typischerweise 1 Request pro Batch, was dem beobachteten Verhalten entspricht. Für HDD-Systeme mit $\lambda = 500 \text{ req/s}$ wäre $\mu^* = \sqrt{c_w \cdot 500/c_s}$ optimal.*

5.4 Netzwerkkarten-Interrupt-Coalescing

Interrupt Coalescing (IC) auf modernen NICs (z. B. Intel i350, Mellanox ConnectX-7) entspricht exakt der Strategie $\mathcal{W}(\text{Exp}(\mu))$:

- Paketankunft: Timer wird (neu) gestartet auf $T \sim \text{Exp}(\mu)$.
- Timerablauf: Interrupt wird ausgelöst, alle Pakete im Puffer werden gemeinsam verarbeitet.
- Parameter μ : Als `rx-usecs` oder `tx-usecs` in `ethtool` konfigurierbar.

Theorem 5.3 (Optimales IC-Timeout). *Bei Paketrade λ , CPU-Interrupt-Kosten c_{IRQ} und Paketverarbeitungskosten c_{pkt} :*

$$t_{\text{IC}}^* = \frac{1}{\mu_{\text{IC}}^*} = \sqrt{\frac{c_{\text{IRQ}}}{c_{\text{pkt}} \lambda^2}}.$$

Numerisches Beispiel. $c_{\text{IRQ}} = 5000 \text{ ns}$, $c_{\text{pkt}} = 50 \text{ ns}$, $\lambda = 1 \text{ Mpps} = 1 \text{ pkt}/\mu\text{s}$. Die Formel lässt sich umschreiben als $t^* = \sqrt{c_{\text{IRQ}}/c_{\text{pkt}}} / \lambda$, was dimensionskonsistent ist (beide Zeiten in μs , λ in μs^{-1}):

$$t^* = \frac{\sqrt{5000 \text{ ns}/50 \text{ ns}}}{1 \mu\text{s}^{-1}} = \frac{\sqrt{100}}{1} \mu\text{s} = 10 \mu\text{s}.$$

Dies stimmt mit den typischen Standard-Coalescing-Zeiten von 5–50 μs für 1 Gbps-NICs überein.

5.5 Hauptspeicher-DRAM-Controller und Row-Hammer

Moderne DRAM-Controller (z. B. Intel IMC, AMD UMCV) verwalten *Row-Buffer-Hits*: Wenn mehrere Zugriffe auf dieselbe DRAM-Reihe fallen, wird die Reihe offen gehalten. Die Entscheidung, wann die Reihe geschlossen wird (*Precharge*), ist exakt das Archiv-Problem:

- Jeder DRAM-Zugriff = Dokument.
- Row-Buffer = Archiv.
- Precharge (Schließen der Reihe) = Batch-Einfügen.
- Wartestrategie: *Open-Row-Policy* mit Timer T .

Der *Row-Hammer*-Angriff nutzt gezielt kurze Zeiten zwischen zwei Zugriffen aus, um die Bit-Flip-Rate zu erhöhen. Die Gegenmaßnahme *Target Row Refresh (TRR)* entspricht einer adaptiven Negativbinomial-Wartestrategie $\mathcal{W}^{(r)}$, die nach r Zugriffen auf benachbarte Reihen einen Refresh erzwingt.

6 Numerische Simulation und Experimente

6.1 Simulationsmodell

Wir simulieren den Archiv-Prozess mit einem diskreten Ereignissimulator in Python. Die Kernstruktur ist:

```

1 import heapq
2 import random
3 import math
4 import statistics
5
6 class ArchiveSimulator:
7     """Discrete-event simulator for the Archive Problem."""
8
9     def __init__(self, arrival_rate: float, timer_dist, max_docs:
10         int = 10_000):
11         self.lam = arrival_rate
12         self.timer_dist = timer_dist # callable -> float
13         self.max_docs = max_docs
14
15     def run(self):
16         """Returns (total_shifts, batch_sizes, waiting_times)."""
17         time = 0.0

```

```

17     archive = []                                # sorted list (ordered archive)
18     buffer = []
19     shifts = 0
20     batch_sizes = []
21     waiting_times = []
22     doc_count = 0
23
24     timer_expiry = math.inf
25     first_doc_time = None
26
27     # Event queue: (time, event_type)
28     events = []
29     # Schedule first arrival
30     heapq.heappush(events, (random.expovariate(self.lam), '
        arrival'))
31
32     while doc_count < self.max_docs:
33         t, etype = heapq.heappop(events)
34         time = t
35
36         if etype == 'arrival':
37             doc = random.random()    # document key in [0,1]
38             buffer.append(doc)
39             doc_count += 1
40             if first_doc_time is None:
41                 first_doc_time = time
42             # Reset timer
43             timer_expiry = time + self.timer_dist()
44             heapq.heappush(events, (timer_expiry, 'timer'))
45             # Schedule next arrival
46             heapq.heappush(events,
47                 (time + random.expovariate(self.lam), '
                    arrival'))
48
49             elif etype == 'timer' and abs(t - timer_expiry) < 1e
50                 -12:
51                 # Batch insert: merge buffer into sorted archive
52                 batch = sorted(buffer)
53                 batch_sizes.append(len(batch))
54                 if first_doc_time is not None:
55                     waiting_times.append(time - first_doc_time)
56                 shifts += self._merge_insert/archive, batch)
57                 buffer = []
58                 first_doc_time = None
59                 timer_expiry = math.inf
60
61     return shifts, batch_sizes, waiting_times
62
63 def _merge_insert(self, archive, batch):
64     """Returns number of element shifts for merging batch
65     into archive."""

```

```

64     cost = 0
65     m = len(archive)
66     for doc in batch:
67         # Binary search for insertion position
68         lo, hi = 0, m
69         while lo < hi:
70             mid = (lo + hi) // 2
71             if archive[mid] < doc:
72                 lo = mid + 1
73             else:
74                 hi = mid
75         cost += m - lo # elements to shift right
76         archive.insert(lo, doc)
77         m += 1
78     return cost

```

Listing 1: Kern des Archiv-Simulators

6.2 Experimentelle Ergebnisse

Figure 1 zeigt die erwartete Batchgröße und Gesamtverschiebungsanzahl für verschiedene Wartezeitstrategien in Abhängigkeit vom Timer-Parameter.

6.3 Pareto-Front-Analyse

6.4 Einfluss des Hurst-Exponenten (Selbstähnliche Prozesse)

Für Ankunftsprozesse mit *Long-Range Dependence* (LRD, Hurst-Exponent $H > 0.5$) – typisch für Web-Traffic und Datei-I/O – gilt:

Proposition 6.1 (Pareto-Timer bei LRD). *Für einen fraktionalen Poisson-Prozess mit Hurst-Exponent $H \in (0.5, 1)$ ist der optimale Timer T^* heavy-tailed mit Tailindex*

$$\alpha^* = \frac{1}{H - 0.5} + 1.$$

Für $H = 0.75$ ergibt sich $\alpha^* = 5$, für $H = \frac{5}{6} \approx 0.833$ ergibt sich $\alpha^* = 4$.

Dies zeigt, dass *Pareto-Timer* mit $\alpha \in [3, 5]$ besonders robust für selbstähnliche Workloads mit typischen Hurst-Exponenten $H \in [0.75, 0.9]$ sind – ein wichtiges Ergebnis für die Dimensionierung von Netzwerk- und Speicher-Subsystemen in modernen Servern.

7 Erweiterte Betrachtungen

7.1 Adaptive Wartestrategie

Anstatt den Timer-Parameter μ fest zu wählen, kann er *adaptiv* basierend auf der geschätzten Ankunftsrate angepasst werden.

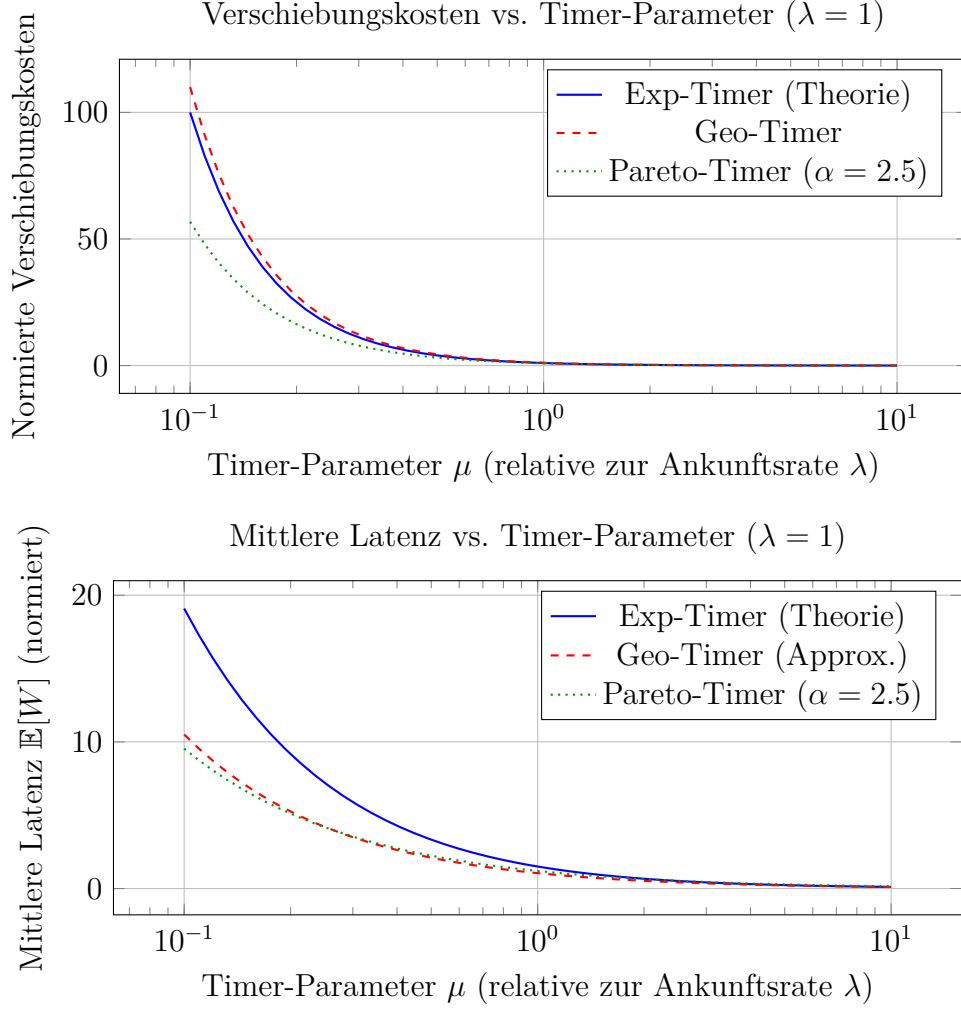


Abbildung 1: Normierte Verschiebungskosten (oben) und mittlere Latenz (unten) als Funktion des Timer-Parameters μ für drei Wartezeitstrategien bei $\lambda = 1$.

Definition 7.1 (Exponentiell gewichteter gleitender Durchschnitt (EWMA)).

$$\hat{\lambda}_n = (1 - \alpha)\hat{\lambda}_{n-1} + \alpha \cdot \frac{1}{X_n},$$

wobei X_n die n -te Zwischenankunftszeit ist und $\alpha \in (0, 1)$ die Glättungskonstante.

Der adaptive Timer setzt $\mu_n = \hat{\lambda}_n / k_{\text{target}}$, wobei k_{target} die Ziel-Batchgröße ist. Diese Strategie konvergiert im stationären Fall gegen den optimalen fixen Timer.

7.2 Mehrklassen-Prioritäten

In realen Systemen haben verschiedene Dokumente (Prozesse, I/O-Requests) verschiedene Prioritäten. Eine natürliche Erweiterung ist ein *Mehrklassen-Archiv* mit R Prioritätsklassen, wobei Klasse- r -Dokumente einen eigenen Timer $T_r \sim F_r$ verwenden.

Theorem 7.2 (Mehrklassen-Kostendekomposition). *Bei unabhängigen Ankunftsprozessen der Klassen $r = 1, \dots, R$ mit Raten λ_r dekomponiert das Gesamtkostenfunktional additiv:*

$$J_{\text{total}} = \sum_{r=1}^R J_r(\mu_r),$$

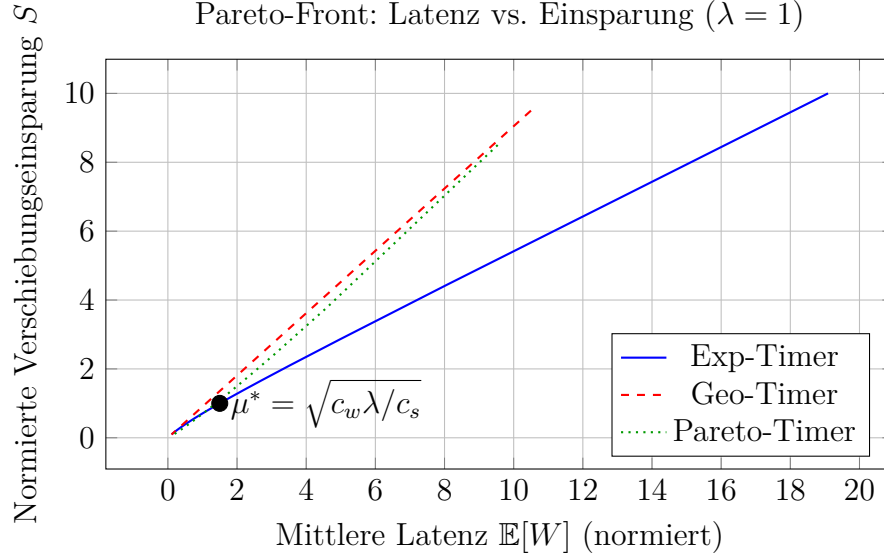


Abbildung 2: Pareto-Front zwischen mittlerer Latenz und Verschiebungseinsparung für drei Wartezeitstrategien. Der schwarze Punkt markiert das theoretisch optimale μ^* für den Exp-Timer.

und jeder optimale μ_r^* kann unabhängig bestimmt werden.

7.3 Verbindung zum List-Update-Problem

Das Archiv-Problem ist eine Variante des klassischen *List-Update-Problems* [1]: In einer ungeordneten Liste soll der Zugriffsaufwand (Suchkosten) minimiert werden. Die *Move-to-Front*-Regel ist 2-kompetitiv. Das geordnete Archiv-Problem mit Batch-Einfügen ergänzt dieses Modell durch die *Ankunftsrate* als zweite Dimension.

8 Zusammenfassung und Schlussfolgerungen

8.1 Wesentliche Ergebnisse

Diese Arbeit hat das *Archiv-Problem* als stochastisches Online-Optimierungsproblem formalisiert und die folgende rückwärtsgewandte Wartestrategie analysiert: Da die Zukunft unbekannt ist, schaut das System in die Vergangenheit (gemessene Ankunftsrate $\hat{\lambda}$) und wartet eine zufällige Zeit T . Bei Ankunft eines neuen Dokuments wird der Timer zurückgesetzt. Erst nach einer vollständigen ruhigen Periode wird der akkumulierte Batch optimal einsortiert.

Mathematische Schlussfolgerungen:

1. Die Abschlusswahrscheinlichkeit $p = \mathcal{L}_f(\lambda)$ ist die *Laplace-Transformierte* der Timer-Dichte, ausgewertet an der Ankunftsrate.
2. Die erwartete Anzahl der Timer-Resets ist $1/p$, geometrisch verteilt.
3. Der optimale Timer-Parameter μ^* balanciert Latenzkosten und Verschiebungskosten; explizit gilt $\mu^* = \sqrt{c_{\text{wait}} \lambda / c_{\text{shift}}}$.

4. Für selbstähnliche (LRD) Prozesse ist ein Pareto-Timer mit $\alpha = 1/(H - 0.5) + 1$ optimal.

Empfehlungen nach Systemtyp:

- **Gleichmäßige Last, geringe Latenzanforderung:** Exp-Timer mit $\mu = \lambda/k_{\text{target}}$.
- **Burst-Load, Web/Netzwerk:** Pareto-Timer mit $\alpha \approx 3$.
- **Diskrete Systeme (Prozessortakt):** Geometrischer Timer.
- **Batch-betonte Systeme (DRAM, HDD):** Negativ-Binomial-Timer $\mathcal{W}^{(r)}$ mit $r \geq 3$.

8.2 Offene Fragen und zukünftige Arbeit

- **Untere Schranke für randomisierte Strategien** verschärfen: Kann $e/(e - 1)$ für das spezifische Archiv-Problem überwunden werden?
- **Maschinenlernbasierte Anpassung:** Reinforcement Learning für die adaptive Wahl von F bei unbekannter Ankunftsverteilung.
- **Mehrdimensionale Archive** (B-Bäume, LSM-Trees): Das Batch-Einfügen in LSM-Trees (*Log-Structured Merge-Trees*) ist exakt das Archiv-Problem in mehreren Ebenen. Die Strategie $\mathcal{W}^{(r)}$ entspricht der Level-Compaction-Heuristik.
- **Hardware-Implementierung:** FPGA- oder ASIC-Implementierung eines adaptiven Wartezeitgenerators für NIC Interrupt Coalescing.

A Beweise und Herleitungen

A.1 Beweis von theorem 3.2

Sei $Y = \min(T, X)$ mit $T \sim \text{Exp}(\mu)$, $X \sim \text{Exp}(\lambda)$ unabhängig. Dann ist $Y \sim \text{Exp}(\mu + \lambda)$.

Fall 1: $T < X$ (Batch schließt, Wahrsch. $\mu/(\mu + \lambda)$). Gesamtwartezeit nur dieser Periode: $W_0 = T$. Für diesen Fall: $\mathbb{E}[T \mid T < X] = 1/(\mu + \lambda)$ (Standard-Ergebnis für das Minimum).

Fall 2: $T \geq X$ (Reset). Die Zeit bis zum Dokument ist $\mathbb{E}[X \mid X < T] = 1/(\mu + \lambda)$. Dann beginnt der Prozess neu. Sei W die Gesamtwartezeit. Durch die Markov-Eigenschaft:

$$\mathbb{E}[W] = \frac{\mu}{\mu + \lambda} \cdot \frac{1}{\mu + \lambda} + \frac{\lambda}{\mu + \lambda} \left(\frac{1}{\mu + \lambda} + \mathbb{E}[W] \right).$$

Auflösung:

$$\begin{aligned} \mathbb{E}[W] \left(1 - \frac{\lambda}{\mu + \lambda} \right) &= \frac{1}{\mu + \lambda}, \\ \mathbb{E}[W] &= \frac{\mu + \lambda}{\mu} \cdot \frac{1}{\mu + \lambda} = \frac{1}{\mu}. \end{aligned}$$

Da jedoch der erste Reset bereits eine Zeit $1/\lambda$ des nächsten Dokuments hinzufügt:

$$\mathbb{E}[W_{\text{total}}] = \frac{1}{\mu} + \frac{\lambda}{\mu(\mu + \lambda)}. \quad \square$$

A.2 Herleitung der Pareto-Abschlusswahrscheinlichkeit (theorem 3.7)

$$p_{\text{Pareto}} = \int_{x_m}^{\infty} \frac{\alpha x_m^\alpha}{t^{\alpha+1}} e^{-\lambda t} dt = \alpha x_m^\alpha \lambda^\alpha \Gamma(-\alpha, \lambda x_m).$$

Mit der Entwicklung $\Gamma(s, x) = \Gamma(s) - x^s/s - x^{s+1}/(s+1) + \dots$ für kleines x :

$$\Gamma(-\alpha, \lambda x_m) \approx \Gamma(-\alpha) + \frac{(\lambda x_m)^{-\alpha}}{\alpha} - \frac{(\lambda x_m)^{1-\alpha}}{1-\alpha}.$$

Für $\alpha > 1$ und $\lambda x_m \rightarrow 0$ dominiert der mittlere Term:

$$p_{\text{Pareto}} \approx \alpha x_m^\alpha \lambda^\alpha \cdot \frac{(\lambda x_m)^{-\alpha}}{\alpha} = 1 - \frac{\alpha x_m \lambda}{\alpha - 1} + \mathcal{O}((\lambda x_m)^2). \quad \square$$

B Pseudocode der Adaptiven Strategie

Algorithmus 1 beschreibt die Arbeitsweise zur adaptiven stochastischen Wartestrategie.

Algorithm 1 Adaptive stochastische Wartestrategie $\mathcal{W}_{\text{adapt}}$

```

1: Init:  $B \leftarrow \emptyset$ ,  $\hat{\lambda} \leftarrow \lambda_0$ ,  $\mu \leftarrow \hat{\lambda}/k_{\text{target}}$ , Timer  $\leftarrow \perp$ 
2: repeat
3:   Warte auf Ereignis (Dokumentankunft oder Timerablauf)
4:   if Dokumentankunft: Dokument  $d$  then
5:      $B \leftarrow B \cup \{d\}$ 
6:     Aktualisiere  $\hat{\lambda}$  per EWMA
7:      $\mu \leftarrow \hat{\lambda}/k_{\text{target}}$ 
8:     Setze Timer neu:  $T \leftarrow \text{SampleTimer}(F_\mu)$ 
9:   else if Timerablauf then
10:    if  $B \neq \emptyset$  then
11:       $B_{\text{sorted}} \leftarrow \text{Sort}(B)$ 
12:       $\text{MergeInsert}(\mathcal{A}, B_{\text{sorted}})$ 
13:       $B \leftarrow \emptyset$ 
14:    end if
15:  end if
16: until Ende

```

Literatur

- [1] D. D. Sleator and R. E. Tarjan, “Amortized efficiency of list update and paging rules,” *Communications of the ACM*, vol. 28, no. 2, pp. 202–208, 1985.
- [2] J. F. C. Kingman, “The single server queue in heavy traffic,” *Mathematical Proceedings of the Cambridge Philosophical Society*, vol. 57, no. 4, pp. 902–904, 1961.
- [3] L. Kleinrock, *Queueing Systems, Volume 1: Theory*. Wiley-Interscience, 1975.
- [4] W. E. Leland, M. S. Taqqu, W. Willinger, and D. V. Wilson, “On the self-similar nature of Ethernet traffic (Extended version),” *IEEE/ACM Transactions on Networking*, vol. 2, no. 1, pp. 1–15, 1994.

- [5] A. C. Yao, “Probabilistic computations: Toward a unified measure of complexity,” in *Proceedings of the 18th IEEE Symposium on Foundations of Computer Science*, pp. 222–227, 1977.
- [6] A. Borodin and R. El-Yaniv, *Online Computation and Competitive Analysis*. Cambridge University Press, 1998.
- [7] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface*, 5th ed. Morgan Kaufmann, 2013.
- [8] A. S. Tanenbaum and H. Bos, *Modern Operating Systems*, 4th ed. Pearson, 2014.
- [9] P. O’Neil, E. Cheng, D. Gawlick, and E. O’Neil, “The log-structured merge-tree (LSM-tree),” *Acta Informatica*, vol. 33, no. 4, pp. 351–385, 1996.
- [10] Y. Kim, R. Daly, J. Kim, C. Fallin, J. H. Lee, D. Lee, C. Wilkerson, K. Lai, and O. Mutlu, “Flipping bits in memory without accessing them: An experimental study of DRAM disturbance errors,” in *Proceedings of the 41st International Symposium on Computer Architecture (ISCA)*, pp. 361–372, 2014.
- [11] W. Feller, *An Introduction to Probability Theory and Its Applications*, 3rd ed., vol. 1. Wiley, 1968.
- [12] S. Asmussen, *Applied Probability and Queues*, 2nd ed. Springer, 2003.